

< Platforma web bazata pe Inteligenta Artificiala
(AI) pentru recomandarea personalizata a
animalelor de companie si suport interactiv post-
adoptie >

Documentul de proiectare

Cuprins

1. Introducere.....	1
1.1 Scopul documentului	1
2. Prezentare generală și abordări de proiectare	2
2.1 Prezentare generală	2
2.2 Presupuneri/ Constrângeri/ Riscuri	2
2.2.1 Presupuneri	2
2.2.2 Constrângeri.....	2
2.2.3 Riscuri	3
3. Considerații de proiectare.....	3
3.1 Obiective și linii directoare (ghiduri)	4
3.2 Metode de dezvoltare	4
3.3 Strategii de arhitectură.....	4
4. Arhitectura Sistemului și Proiectarea Arhitecturii	5
4.1 Vedere logică.....	6
4.2 Arhitectură hardware	6
4.3 Arhitectură software.....	6
4.4 Arhitectura informațiilor	7
4.5 Arhitectura de comunicații interne	8
4.6 Diagrama de arhitectură a sistemului.....	8
5. Proiectarea sistemului	8
5.1 Proiectarea bazei de date	9
5.1.1 Obiecte de date și structuri de date rezultante	9
5.1.2 Fișiere și baze de date	9
5.2 Conversii de date.....	10
5.3 Interfețe utilizator	10
5.3.1 Intrări	10
5.3.2 ieșiri.....	10
5.4 Proiectarea interfețelor cu utilizatorul	Error! Bookmark not defined.
6. Scenarii de utilizare	12
7. Proiectare de detaliu	12
7.1 Proiectare hardware de detaliu	13
7.2 Proiectare software de deatliu	13
7.3 Proiectare detaliată de securitate.....	15
7.4 Proiectare de detaliu pentru performanța sistemului	15
7.5 Proiectare detaliată a comunicațiilor interne (între componente).....	16
8. Controale pentru verificarea integrității sistemului.....	18

Anexa A: Gestiunea modificărilor documentului.....	Error! Bookmark not defined.
Anexa B: Acronime.....	Error! Bookmark not defined.
Anexa C Documente la care se face referire.....	Error! Bookmark not defined.

1. Introducere

Acest Document de Proiectare a Sistemului (DPS) detaliază arhitectura tehnică a platformei web AI destinate adopției și suportului post-adopție pentru animalele de companie. Documentul transformă cerințele funcționale și non-funcționale în specificații tehnice clare.

Acesta descrie proiectarea la nivel înalt și de detaliu, incluzând structura bazei de date, stack-ul tehnologic (client-server), integrarea modelelor AI (matchmaking și chatbot) și interfața utilizatorului.

Fiind un document dinamic, DPS va evolua pe parcursul implementării. Deoarece conține scheme tehnice detaliate și fluxuri de autentificare, documentul are caracter confidențial, fiind destinat exclusiv evaluării academice pentru a proteja securitatea viitorului sistem.

1.1 Scopul documentului

Scopul acestui Document de Proiectare a Sistemului este de a defini riguros arhitectura și designul platformei web AI dedicate adopției animalelor de companie. El transpune cerințele proiectului într-o foaie de parcurs tehnică clară, ghidând procesul de implementare pentru modulele de matchmaking și asistent virtual.

Documentul este elaborat iterativ pe parcursul realizării licenței. Publicul țintă este format din studentul dezvoltator, profesorul coordonator și comisia de evaluare. Totodată, secțiunile vizuale, precum interfața utilizatorului (UI), pot fi prezentate reprezentanților ONG-urilor sau potențialilor adoptatori pentru validare și feedback.

2. Prezentare generală și abordări de proiectare

Această secțiune descrie principiile și strategiile care vor fi utilizate ca ghiduri în momentul proiectării și implementării sistemului.

2.1 Prezentare generală

Această secțiune oferă o perspectivă de ansamblu asupra arhitecturii platformei. Proiectarea urmează o abordare modulară, de tip client-server, separând clar interfața cu utilizatorul (frontend) de logica de procesare a datelor (backend).

Obiectivul principal de proiectare este crearea unui sistem scalabil, robust și ușor de întreținut, capabil să integreze eficient modulele de Inteligență Artificială pentru procesul de matchmaking și asistentul virtual. Arhitectura software se bazează pe servicii web (API REST) care facilitează o comunicare fluidă și securizată între interfața web interactivă, baza de date a adăposturilor și modelele externe de AI, garantând astfel o experiență optimă pentru utilizator.

2.2 Presupuneri/ Constrângeri/ Riscuri

2.2.1 Presupuneri

Pentru proiectarea platformei s-au definit următoarele presupuneri și dependențe esențiale:

La nivel de utilizare, se presupune că adoptatorii și reprezentanții ONG-urilor dețin cunoștințe digitale de bază și dispun de o conexiune stabilă la internet. Din punct de vedere hardware și software, sistemul nu necesită echipamente dedicate, rulând optim pe orice sistem de operare standard prin intermediul browserelor web moderne.

La nivel tehnic, funcționalitatea de bază a platformei depinde critic de disponibilitatea continuă a serviciilor cloud de găzduire și a API-urilor externe de Inteligență Artificială (utilizate pentru chatbot și matchmaking). De asemenea, se presupune că adăposturile partenere vor menține constant actualizate profilurile animalelor în baza de date.

2.2.2 Constrângeri

Proiectarea și implementarea platformei sunt supuse unor constrângeri majore care influențează direct deciziile de arhitectură. În primul rând, deoarece sistemul colectează date personale despre stilul de viață, locuința și programul utilizatorilor pentru a realiza potrivirea optimă cu animalele, acesta intră sub incidența directă a regulamentelor privind protecția datelor (GDPR). Această constrângere legislativă și de securitate impune ca designul bazei de date să includă obligatoriu criptarea informațiilor sensibile. De asemenea, arhitectura trebuie să susțină funcționalități de ștergere completă a istoricului și profilului, ceea ce necesită o gestionare atentă a relațiilor din baza de date pentru a evita orfanizarea informațiilor.

În al doilea rând, există constrângeri tehnice legate de interoperabilitate și de dependența de serviciile externe. Funcționalitățile principale ale platformei, cum ar fi algoritmul de

matchmaking și asistentul virtual post-adopție, depind de modele de inteligență artificială terțe. Aceste API-uri impun limite de utilizare și depind de latența rețelei de internet. Prin urmare, arhitectura software trebuie concepută în mod asincron, astfel încât sistemul să gestioneze elegant eventualele întârzieri sau indisponibilități ale acestor servicii, menținând interfața activă și afișând mesaje de așteptare corespunzătoare.

Din punct de vedere al infrastructurii, fiind vorba despre un proiect de licență, platforma utilizează medii de găzduire cloud în limitele pachetelor gratuite sau educaționale. Acestea vin cu restricții stricte privind capacitatea de memorie, spațiul de stocare și lățimea de bandă. Acest lucru are un impact direct asupra implementării, impunând o optimizare severă a codului backend și a interogărilor către baza de date. Totodată, imaginile încărcate de reprezentanții adăposturilor trebuie comprimate automat pentru a nu epuiza spațiul alocat pe server.

Nu în ultimul rând, mediul utilizatorului final impune constrângeri clare de design. Adoptatorii și voluntarii aflați pe teren vor accesa aplicația de pe o multitudine de dispozitive mobile, folosind adesea conexiuni de date fluctuante. Astfel, interfața grafică este constrânsă să urmeze o abordare de tip „mobile-first”, asigurând un design complet responsiv și elemente vizuale suficient de ușoare pentru a garanta o performanță optimă la încărcare.

2.2.3 Riscuri

Principalul risc al platformei este generarea unor sfaturi eronate de către asistentul virtual AI, ceea ce ar putea pune în pericol sănătatea animalului. Pentru atenuare, sistemul va integra filtre stricte bazate pe cuvinte cheie și mesaje de avertizare, direcționând utilizatorul către medicul veterinar în caz de urgențe medicale. Un alt risc tehnic major este indisponibilitatea temporară a API-urilor externe, care ar putea bloca funcțiile de bază. Strategia de reducere constă în implementarea unor mecanisme asincrone de gestionare a erorilor, care să mențină aplicația activă. De asemenea, riscul scurgerilor de date este diminuat prin criptarea avansată a bazei de date.

3. Considerații de proiectare

Instrucțiuni: Descrieți problemele care trebuie abordate sau rezolvate înainte de a încerca să elaborați o soluție de design completă.

3.1 Obiective și linii directoare (ghiduri)

Proiectarea platformei este ghidată de obiectivul principal de a oferi o experiență de utilizare empatică, intuitivă și extrem de accesibilă. Interfața vizuală este concepută minimalist, inspirată de aplicațiile moderne de tip „lifestyle”, punând accent pe claritate și navigare facilă, astfel încât utilizatori de toate vârstele să poată parcurge procesul de adopție fără dificultăți tehnice. Din perspectiva performanței, s-a decis prioritizarea vitezei de răspuns a interfeței în detrimentul unui consum marginal mai mare de memorie pe dispozitivul clientului. Acest lucru se realizează prin utilizarea tehnicilor de preîncărcare a resurselor și stocare locală, garantând o interacțiune fluidă, în special în timpul conversațiilor în timp real cu asistentul virtual.

La nivel de implementare a codului, dezvoltarea urmează riguros principiile arhitecturii modulare și ale codului curat, precum evitarea redundanței (DRY - Don't Repeat Yourself). Convențiile de denumire a funcțiilor, variabilelor și structurarea endpoint-urilor respectă standardele RESTful. Motivul principal pentru aceste linii directoare este asigurarea unei mentenanțe ușoare și facilitarea extinderii viitoare a platformei. În ceea ce privește modulul de inteligență artificială, o direcție esențială este calibrarea atentă a personalității asistentului virtual prin ingineria prompturilor, astfel încât răspunsurile generate să adopte mereu un ton cald, încurajator, dar responsabil și ferm în privința limitărilor medicale.

O altă strategie de design care influențează direct implementarea este preferința pentru utilizarea unor biblioteci open-source consacrate și a componentelor vizuale predefinite pentru elementele standard ale interfeței. Această alegere nu modifică arhitectura de bază a sistemului, dar optimizează semnificativ timpul de dezvoltare, permițând concentrarea eforturilor tehnice pe complexitatea algoritmilor de matchmaking și pe protocoalele de siguranță ale aplicației.

3.2 Metode de dezvoltare

Proiectarea sistemului urmează o abordare Agile, utilizând prototiparea iterativă pentru a rafina funcționalitățile AI. Arhitectura software este un monolit modular organizat după principii orientate pe obiecte, utilizând framework-uri bazate pe componente (ex. React) pentru frontend și o structură clară de tip rute/servicii pentru backend.

Pentru documentarea tehnică și modelarea fluxurilor de date asincrone, am utilizat standardul UML (diagrame de cazuri de utilizare și de secvență). Am ales această structură în detrimentul microserviciilor pentru a reduce complexitatea și a optimiza viteza de dezvoltare în contextul resurselor limitate.

Ca plan de rezervă pentru riscul instabilității API-urilor externe de AI, am implementat un șablon de proiectare de tip Adapter. Acesta permite comutarea rapidă între furnizorii de modele lingvistice sau trecerea la un sistem de suport bazat pe reguli fixe, fără a rescrie logica întregii platforme.

3.3 Strategii de arhitectură

Strategia de arhitectură a platformei se bazează pe câteva decizii fundamentale menite să echilibreze inovația AI cu stabilitatea necesară unei aplicații web moderne.

Arhitectura de tip Single Page Application (SPA) cu React și Node.js

S-a ales utilizarea framework-ului React pentru frontend și Node.js pentru backend.

Raționamentul principal este uniformitatea limbajului (JavaScript/TypeScript) pe ambele niveluri, ceea ce accelerează dezvoltarea. Am optat pentru o arhitectură SPA pentru a oferi o experiență de utilizare fluidă, similară unei aplicații native, esențială pentru interacțiunea cu chatbot-ul. S-a luat în considerare utilizarea unui model tradițional de randare pe server (Server-Side Rendering), dar a fost respins deoarece ar fi crescut latența la fiecare interacțiune a utilizatorului, afectând negativ fluxul conversațional al AI-ului.

Gestionarea datelor cu MongoDB (NoSQL)

Pentru persistența datelor, am selectat MongoDB. Această decizie a fost dictată de natura variabilă a profilurilor animalelor (care pot avea seturi diferite de caracteristici, istoric medical sau comportamental) și a datelor colectate din conversațiile chatbot-ului. O bază de date relațională (cum ar fi PostgreSQL) a fost considerată, însă structura sa rigidă ar fi îngreunat evoluția rapidă a modelelor de date necesare pentru algoritmul de matchmaking AI.

Integrarea asincronă a serviciilor AI și Pattern-ul Fațadă

O strategie cheie este izolarea interacțiunii cu API-urile externe (OpenAI/Gemini) prin mecanisme asincrone și utilizarea unui „Adapter” sau a unei „Fațade”. Aceasta permite sistemului să nu rămână blocat în timp ce așteaptă răspunsul de la modelele de limbaj mari. Raționamentul este legat de prioritatea acordată disponibilității: dacă serviciul extern are latență mare, sistemul afișează un indicator de procesare și permite utilizatorului să navigheze în alte secțiuni. Această abordare compensează riscul dependenței de terți prin flexibilitate arhitecturală.

Planuri de extindere și mentenanță

Sistemul este proiectat să fie stateless (fără stare pe server), utilizând token-uri JWT pentru autentificare. Această strategie permite o scalare facilă în viitor (adăugarea mai multor instanțe de server fără a pierde sesiunile utilizatorilor). În plus, designul modular permite adăugarea ulterioară a unor module de tele-medicină veterinară sau sisteme de monitorizare IoT fără a restructura nucleul platformei.

4. Arhitectura Sistemului și Proiectarea Arhitecturii

Arhitectura sistemului urmează modelul Client-Server, fiind descompusă în trei straturi principale pentru a asigura o separare clară a responsabilităților:

- Frontend (Client): Gestionează interfața utilizatorului și colectarea datelor, comunicând asincron cu serverul prin JSON.
- Backend (Server API): Orchestrează logica de business, autentificarea și fluxul de date între baza de date și serviciile externe.
- Modulul AI (Adapter): Izolează interacțiunea cu API-urile externe (OpenAI), transformând cererile interne în formate compatibile cu modelele de limbaj.

Această structură utilizează tiparul Facade pentru a simplifica accesul frontend-ului la resurse și tiparul Adapter pentru a proteja sistemul de modificările serviciilor AI terțe. Datele circulă fluid prin endpoint-uri REST, asigurând o decuplare care permite mentenanța ușoară fără a afecta integritatea întregului sistem.

4.1 Vedere logică

Vizarea logică a sistemului este reprezentată printr-un set de diagrame UML, care mapează structura internă și comportamentul platformei. Acestea includ:

- Diagrama de Clase: Definește entitățile principale (Utilizator, Animal, Adăpost, Mesaj) și relațiile dintre ele, evidențiind modul în care datele sunt structurate în obiecte programabile.
- Diagrama de Cazuri de Utilizare (Use Case): Ilustrează interacțiunile funcționale dintre actori (Adoptator, Administrator ONG) și sistem, definind granițele aplicației.
- Diagrame de Secvență: Detaliază fluxul logic al operațiunilor complexe, cum ar fi procesul de matchmaking, descriind ordinea apelurilor între frontend, backend și API-ul AI.

4.2 Arhitectură hardware

Sistemul utilizează o arhitectură distribuită în cloud, fiind structurat pe instanțe virtuale ce separă nivelurile de prezentare, aplicație și date pentru o performanță optimă. Frontend-ul este livrat printr-o rețea de tip CDN, în timp ce logica de backend rulează pe un server virtual dotat cu minimum 2 vCPUs și 4GB RAM, resurse suficiente pentru procesarea fluxurilor AI. Datele sunt gestionate într-un cluster extern securizat, accesibil serverului de aplicație doar prin rețeaua privată virtuală, în timp ce traficul extern este strict filtrat de un firewall și protejat prin protocolul HTTPS pe portul 443. Această configurație, completată de un spațiu de stocare de 50GB pentru resurse media, asigură o infrastructură scalabilă, centralizată logic, dar distribuită fizic pentru siguranță.

4.3 Arhitectură software

Arhitectura software a platformei este construită pe stack-ul tehnologic MERN (MongoDB, Express, React, Node.js), utilizând limbajul TypeScript pentru a asigura rigoarea tipizării și o mentenanță facilă. Stratul de prezentare este dezvoltat în React 18, folosind componente funcționale și hook-uri pentru gestionarea stării, în timp ce logica de aplicație rulează pe un server Node.js 22 cu framework-ul Express, oferind o procesare asincronă eficientă a cererilor. Persistența datelor este gestionată prin MongoDB Atlas, o platformă NoSQL care permite stocarea flexibilă a profilurilor de animale sub formă de documente JSON. Comunicarea între componente se realizează exclusiv prin API-uri de tip RESTful, utilizând protocolul HTTPS pentru securizarea schimbului de mesaje.

Sistemul integrează API-ul extern OpenAI (GPT-4o) pentru procesarea limbajului natural în modulele de asistent virtual și matchmaking, interfațarea fiind realizată prin librăria Axios. Din punct de vedere al segmentării, software-ul este împărțit în module logice clar definite: modulul de autentificare, cel de gestiune a bazei de date și motorul de procesare AI, fiecare fiind descompus în clase și funcții specifice care izolează logica de business. Fluxul de date pornește de la input-ul utilizatorului în interfață, este prelucrat de controlerele din backend, trimis către serviciul de inteligență artificială pentru generarea răspunsului și, în final, transformat în ieșirea vizuală afișată adoptatorului. Această structură modulară, bazată pe un design stateless, a fost aleasă pentru a maximiza performanța interacțiunilor în timp real și pentru a permite extinderea facilă a funcționalităților fără a perturba nucleul sistemului.

4.4 Arhitectura informațiilor

Arhitectura informațională a sistemului este structurată pentru a gestiona fluxuri complexe de date necesare procesului de adopție, fiind organizată în trei categorii principale: date de profil, date operaționale și metadate de interacțiune. Informațiile stocate includ profilurile utilizatorilor (nume, date de contact, preferințe), profilurile detaliate ale animalelor (specie, istoric medical, trăsături comportamentale și resurse media) și istoricul de comunicare generat de asistentul virtual. Printre acestea, o pondere semnificativă o reprezintă informațiile cu caracter sensibil, cum ar fi datele de identificare personală (PII) și detaliile despre stilul de viață sau condițiile de locuire ale utilizatorilor. Aceste date sunt considerate sensibile deoarece permit profilarea comportamentală, motiv pentru care sunt protejate prin mecanisme de criptare și acces restricționat conform reglementărilor GDPR.

Toate datele procesate de platformă sunt furnizate exclusiv în format electronic. Principalele surse de informații sunt utilizatorii finali (adoptatorii), care introduc manual datele prin formulare web și chestionare de compatibilitate, și reprezentanții adăposturilor (administratorii), responsabili de introducerea manuală și actualizarea fișelor digitale ale animalelor. Sistemul primește, de asemenea, date electronice automate sub formă de răspunsuri structurate (JSON) de la API-urile modelelor de limbaj (AI), care furnizează analizele de compatibilitate și soluțiile de suport. Deși sursele primare pot proveni uneori din documente fizice (de exemplu, carnete de sănătate în format hârtie deținute de adăposturi), introducerea acestora în sistem se realizează strict prin digitalizare manuală de către operatori.

4.5 Arhitectura de comunicații interne

Arhitectura de comunicații a sistemului SmartAdopt este una virtualizată, bazată pe un mediu de tip Virtual Private Cloud (VPC), care izolează logic resursele aplicației pentru a asigura un flux de date securizat și controlat. Componentele sistemului sunt interconectate printr-o rețea privată definită prin software, unde comunicarea între frontend, backend și baza de date este strict reglementată de reguli de rutare interne.

În loc de echipamente fizice tradiționale, sistemul utilizează echivalente virtuale de înaltă performanță. Un Internet Gateway asigură punctul de intrare pentru traficul extern, în timp ce un Application Load Balancer distribuie cererile către instanțele de backend. Securitatea este garantată de un Cloud Firewall (sau Security Groups), care acționează ca o barieră, permițând accesul public doar pe portul 443 (HTTPS) și blocând orice tentativă de acces extern direct către baza de date, care comunică intern pe portul 27017. Modulele de comunicații utilizează protocolul TLS 1.3 pentru a cripta orice transfer de date între serverul de aplicație și API-urile externe de inteligență artificială.

Fluxul de comunicații urmează o cale liniară: cererea utilizatorului ajunge prin WAN la Load Balancer, este redirectionată către serverul de aplicație în rețeaua privată, iar acesta din urmă interoghează baza de date sau serviciile AI prin tuneluri criptate. Din punct de vedere al resurselor, se estimează o necesitate de bandă WAN de aproximativ 5-10 Mbps pentru a gestiona simultan încărcarea imaginilor și fluxurile de chat. În interiorul rețelei LAN virtuale, lățimea de bandă este garantată de furnizorul cloud la viteze de peste 1 Gbps, asigurând o latență minimă între serverul de aplicație și baza de date, factor critic pentru performanța algoritmului de matchmaking.

4.6 Diagrama de arhitectură a sistemului

Arhitectura sistemului este organizată pe trei niveluri principale, integrate într-un mediu cloud securizat. Primul nivel, cel de Prezentare, constă în interfața React care comunică prin HTTPS cu serverul de Aplicație (Node.js), protejat de un firewall și un load balancer. Serverul centralizează logica de business, interogând Baza de Date (MongoDB) pentru informații persistente și interfațând cu API-ul OpenAI pentru procesarea inteligentă a datelor.

Fluxul informațional este bidirecțional și asincron, utilizând obiecte JSON pentru a asigura o comunicare rapidă între componentele izolate logic. Această structură garantează atât securitatea datelor sensibile prin izolarea bazei de date, cât și o experiență de utilizare fluidă, esențială pentru interacțiunea cu asistentul virtual.

5. Proiectarea sistemului

5.1 Proiectarea bazei de date Proiectarea Bazei de Date

Platforma utilizează MongoDB Atlas ca sistem de gestiune NoSQL, stocând informațiile sub formă de documente flexibile, potrivite pentru datele variate ale animalelor. Resursele media (imagini) sunt gestionate extern într-un serviciu de tip Object Storage, baza de date păstrând doar referințele URL.

Dicționarul de date cuprinde entități esențiale precum Utilizator (ID unic, email securizat), Animal (nume, status adopție, istoric medical) și Interacțiuni (scoruri de matchmaking generate de AI și log-uri de chat). Validarea se realizează riguros în backend (ex: format email, limite de caractere), iar operațiunile CRUD sunt strict ierarhizate: utilizatorii au acces doar la propriul profil, în timp ce administratorii de ONG-uri pot gestiona integral baza de date a animalelor. Performanța este garantată prin indexarea câmpurilor interogate frecvent, asigurând un sistem rapid, scalabil și conform cu cerințele de securitate.

5.1.1 Obiecte de date și structuri de date rezultante

Obiectele de date funcționale ale sistemului sunt traduse în structuri de date flexibile, optimizate pentru procesare asincronă și interacțiuni AI. Profilul Utilizatorului și cel al Animalului sunt structurate ca **Documente BSON** (obiecte JSON îmbogățite), care permit imbricarea datelor complexe, precum sub-obiecte pentru stilul de viață al adoptatorului sau tablouri (arrays) pentru istoricul medical și galeria media a animalului. Această ierarhie asigură o procesare rapidă în backend, eliminând necesitatea unor operațiuni de tip join care ar încetini sistemul.

O structură de date critică pentru funcționarea asistentului virtual este **Obiectul de Context**, o listă ordonată de mesaje de tip tuplu, transmisă între serverul de aplicație și API-ul extern pentru a menține memoria conversației. Rezultatul procesului de matchmaking este încapsulat într-un **Data Transfer Object (DTO)** în format JSON, care conține identificatori unici, scoruri numerice de compatibilitate și argumentări textuale generate de AI. Aceste structuri circulă constant între componente prin apeluri REST, asigurând o decuplare logică și o comunicare fluidă între interfața de prezentare și nucleul de calcul al aplicației.

5.1.2 Fișiere și baze de date

Modelul de date fizic este structurat pe o arhitectură hibridă ce separă înregistrările tranzacționale de fișierele media de mari dimensiuni. Nucleul sistemului, baza de date MongoDB Atlas, este găzduit pe instanțe virtuale cu stocare de tip SSD în centre de date regionale din Europa, asigurând replicarea automată a datelor pe mai multe zone de disponibilitate pentru a preveni pierderile accidentale. Datele nestructurate, precum fotografiile animalelor și documentația medicală scanată, sunt stocate fizic sub formă de obiecte binare (blobs) într-un serviciu de tip Cloud Object Storage, fiind accesibile prin legături securizate stocate în baza de date principală.

La nivelul serverului de aplicație, configurațiile sensibile sunt păstrate în fișiere de mediu protejate, în timp ce jurnalele de sistem (logs) sunt gestionate local printr-o structură de fișiere cu rotație automată pentru a preveni saturarea stocării. Această distribuție fizică asigură o

scalabilitate ridicată, deoarece serverul de aplicație procesează doar metadatele ușoare, în timp ce resursele hardware specializate din cloud preiau sarcina stocării masive și a securizării accesului la informațiile persistente.

5.2 Conversii de date

Conversiile de date în cadrul platformei vizează următoarele procese esențiale:

- Interoperabilitate: Serializarea obiectelor în format JSON pentru comunicarea între frontend și backend.
- Integrare AI: Transformarea datelor structurate din baza de date în text (limbaj natural) pentru prompt-urile AI și, invers, parsarea răspunsurilor textuale ale AI-ului în obiecte cu scoruri numerice.
- Persistență: Conversia automată BSON-Object realizată de driver-ul MongoDB pentru stocarea datelor.
- Media: Transformarea fișierelor binare de imagine în URL-uri de resurse optimizate pentru web.

Documentația completă a acestor transformări este disponibilă în fișierele de utilități din directorul /src/utils/converters al codului sursă.

5.3 Interfețe utilizator

Sistemul deservește trei categorii principale de utilizatori, diferențiați prin responsabilități și mod de acces:

- Adoptator Potențial (Extern): Utilizator cu competențe de bază; folosește interfața publică și asistentul AI pentru matchmaking. (Estimări: 1.000 total / 100 simultan).
- Reprezentant ONG (Extern): Competențe medii; accesează panoul de administrare pentru a gestiona profilurile animalelor și cererile primite. (Estimări: 50 total / 10 simultan).
- Administrator & Suport (Intern): Competențe tehnice ridicate; monitorizează infrastructura, securitatea și oferă asistență operațională. (Estimări: <10 total / 3 simultan).

Accesul este securizat diferențiat: utilizatorii externi interacționează prin internetul public (WAN), în timp ce personalul intern utilizează conexiuni protejate pentru sarcini administrative.

5.3.1 Intrări

Sistemul colectează date prin trei modalități principale: formulare web structurate, o fereastră de chat pentru asistentul AI și un selector de fișiere pentru imagini. Aceste intrări sunt mapate direct către baza de date, unde formularele alimentează profilurile de utilizator și animal, iar chat-ul este transformat în log-uri de conversație.

Fiecare element de date are reguli stricte de validare. Adresa de email trebuie să aibă un format valid și să fie unică în sistem, vârsta utilizatorului trebuie să se încadreze între 18 și 99 de ani, iar numele animalelor trebuie să conțină doar caractere alfanumerice, cu o lungime între 2 și

100 de caractere. Mesajele din chat sunt limitate la 1000 de caractere pentru a optimiza procesarea AI și sunt curățate automat de orice cod malițios.

Pentru a preveni introducerea de date eronate, sistemul folosește o validare dublă: una imediată în interfață (prin mesaje de eroare instantanee) și una finală pe server, care blochează salvarea datelor dacă regulile nu sunt respectate. Utilizatorul primește feedback constant prin mesaje de succes, cum ar fi „Profil salvat cu succes”, sau avertismente specifice, de exemplu „Formatul fișierului nu este acceptat” sau „Adresa de email există deja”. Această abordare elimină riscul de a corupe baza de date și asigură o experiență de utilizare fluidă.

5.3.2 Ieșiri

Ieșirile sistemului SmartAdopt sunt concepute pentru a transforma datele brute în informații vizuale clare, facilitând procesul de luare a deciziilor pentru adoptatori și gestionarea eficientă pentru ONG-uri.

Identificarea și Conținutul Ieșirilor

Principalele ieșiri sunt reprezentate prin ecrane de afișare și rapoarte dinamice, identificate prin următoarele coduri interne:

- **AdoptMatch-01 (Raportul de Matchmaking):** Acesta este rezultatul central al sistemului, afișând un scor numeric de compatibilitate, o argumentare textuală generată de AI și o listă de animale recomandate. Elementele de date includ `match_score` și `pet_name`, extrase direct din baza de date și procesate de motorul AI.
- **UserDash-01 (Panoul Adoptatorului):** O interfață grafică ce centralizează istoricul conversațiilor (`chat_log`), statusul cererilor trimise și galeria de imagini a animalelor salvate la "favorite".
- **ShelterDash-01 (Panou Administrare ONG):** Un ecran de gestiune care oferă vizualizarea statistică a adopțiilor finalizate și lista cererilor primite, incluzând datele de profil ale potențialilor adoptatori.
- **AIChat-01 (Interfața de Dialog):** Fluxul de mesaje returnat în timp real de asistentul virtual, formatat pentru a fi ușor de citit în fereastra de chat a utilizatorului.

Scopul și Utilizatorii Principali

Scopul principal al acestor ieșiri este de a simplifica procesul de selecție a unui animal de companie, reducând timpul de căutare pentru Adoptatori prin recomandări personalizate. Pentru Reprezentanții ONG, aceste ieșiri servesc ca instrumente de filtrare și monitorizare, ajutându-i să prioritizeze cererile cele mai promițătoare. De asemenea, rapoartele permit administratorilor să evalueze performanța algoritmului de matchmaking și gradul de interacțiune pe platformă.

Accesul la aceste ieșiri este strict ierarhizat prin mecanisme de Role-Based Access Control (RBAC). Adoptatorii au acces exclusiv la propriul panou (UserDash-01) și nu pot vizualiza datele altor utilizatori sau rapoartele interne ale adăposturilor. Reprezentanții ONG pot vizualiza doar informațiile legate de animalele din propria gestiune și profilurile utilizatorilor care au aplicat direct pentru acele animale. Toate ieșirile care conțin date cu caracter sensibil sunt transmise prin protocol criptat HTTPS, iar sesiunile sunt protejate prin token-uri de autentificare temporare (JWT), asigurând că informațiile nu sunt interceptate sau accesate neautorizat.

6. Scenarii de utilizare

Funcționalitatea generală a sistemului SmartAdopt se bazează pe medierea inteligentă între dorința de adopție și nevoile specifice ale animalelor din adăposturi, transformând căutarea manuală într-un proces ghidat de AI.

În primul scenariu operațional, cel al Adoptatorului, fluxul începe când utilizatorul accesează interfața de chat și primește stimuli sub formă de întrebări despre stilul său de viață și preferințe. Pe măsură ce utilizatorul răspunde, sistemul procesează aceste intrări prin motorul AI, interoghează baza de date pentru a găsi potriviri și returnează un raport de matchmaking cu recomandări personalizate. Finalizarea acestui scenariu are loc când adoptatorul selectează un profil de animal și apasă butonul de aplicare, acțiune care transferă datele sale către sistemul de gestionare al adăpostului.

Al doilea scenariu vizează Reprezentantul ONG, care interacționează cu platforma pentru a înregistra un animal nou. Fluxul presupune introducerea descrierii și încărcarea fotografiilor, moment în care sistemul analizează automat textul pentru a extrage trăsături comportamentale și a le indexa pentru viitoare potriviri. Atunci când apare o cerere de adopție, reprezentantul primește o notificare de „Match” ridicat, analizează profilul solicitantului prin panoul de control și actualizează statusul animalului în „proces de adopție”, declanșând automat informarea utilizatorului extern.

Din perspectiva Administratorului, sistemul funcționează într-un mod de supraveghere, asigurând integritatea comunicării între API-ul extern și baza de date. Scenariile de întreținere includ monitorizarea alertelor de securitate și gestionarea cotelor de procesare pentru a preveni întreruperea serviciului.

Un aspect critic al proiectării este definirea a ceea ce sistemul nu ar trebui să facă: acesta este programat să refuze recomandările dacă există indicii de incompatibilitate majoră care ar putea duce la abandon și să blocheze orice tentativă de acces la datele de contact personale înainte de validarea oficială a cererii. Toate aceste interacțiuni sunt legate logic prin fluxuri de date criptate, asigurând că nicio informație sensibilă nu părăsește mediul securizat al aplicației fără autorizarea corespunzătoare.

7. Proiectare de detaliu

Pentru a construi și integra sistemul eficient, echipa de dezvoltare trebuie să se concentreze pe următoarele direcții:

1. **Infrastructură și Securitate (Hardware Virtual)**
 - **Izolare:** Configurarea unui mediu cloud (VPC) cu subrețele diferite. Baza de date rămâne în rețeaua privată, inaccesibilă direct din internet.
 - **Control Acces:** Firewall-ul permite doar traficul HTTPS (Port 443) pentru utilizatori și conexiuni interne securizate pentru baza de date.
2. **Dezvoltare Software**
 - **Cod Modular:** Utilizarea TypeScript pentru o structură clară. Logica AI, gestiunea bazei de date și interfața trebuie să fie decuplate (servicii separate).
 - **Autentificare:** Implementarea de token-uri JWT pentru a securiza fiecare interacțiune între frontend și backend.
3. **Integrarea Pachetelor Externe**
 - **Gestiune:** Folosirea unui manager de pachete (npm/yarn) pentru a menține versiuni stabile ale bibliotecilor (OpenAI, React, MongoDB).
 - **Adaptare (Wrapper):** API-ul OpenAI nu va fi apelat direct, ci printr-un serviciu intern dedicat, permițând schimbarea ușoară a modelului AI pe viitor fără a rescrie toată aplicația.
4. **Automatizare și Lansare**
 - **Consistență:** Împachetarea aplicației în containere Docker pentru a asigura funcționarea identică pe computerele dezvoltatorilor și pe serverele de producție.
 - **Flux CI/CD:** Implementarea unui proces automat de testare și livrare la fiecare modificare de cod.

Proiectare hardware de detaliu

Sistemul SmartAdopt este construit pe o arhitectură cloud-native, ceea ce înseamnă că în loc de servere fizice închiriate, folosim resurse virtuale scalabile de la furnizori precum AWS și MongoDB Atlas.

Pentru procesare, utilizăm instanțe AWS EC2 (model t3.medium), care oferă resursele necesare (CPU și RAM) pentru a rula backend-ul aplicației. Baza de date este găzduită pe un cluster MongoDB Atlas (M10), echipat cu stocare SSD rapidă și replicare automată a datelor pentru a preveni pierderile. Imaginile și fișierele media nu sunt ținute pe serverul principal, ci într-un serviciu specializat de stocare de tip obiect, AWS S3, care permite accesul rapid și scalarea nelimitată.

La nivel de rețea, un AWS Application Load Balancer acționează ca poartă de intrare, distribuind traficul și asigurând securitatea prin criptare. Un avantaj major al acestei configurații este că nu necesită hardware specializat pentru AI (cum ar fi unități GPU scumpe), deoarece logica complexă de procesare a limbajului este externalizată prin API către infrastructura OpenAI. Totul este protejat prin criptare hardware standard (AES-256), garantând siguranța datelor fără investiții masive în echipamente fizice.

Proiectare software de detaliu

Proiectarea de detaliu a sistemului SmartAdopt este structurată pe trei servicii software principale care conlucrează pentru a oferi experiența de matchmaking asistată de AI.

1. Serviciul de Autentificare și Securitate (AuthGateway)

Acesta este un serviciu de tip Middleware/Aplicație responsabil de controlul accesului. Scopul său este verificarea identității și autorizarea acțiunilor în funcție de rolul utilizatorului (Adoptator sau ONG). Satisface cerințele de securitate și integritate a datelor.

- **Structuri și Procesare:** Utilizează obiecte de tip JWT Payload (conținând ID-ul și rolul utilizatorului) stocate temporar în memoria cache pentru validare rapidă. Procesarea implică algoritmi de hashing (bcrypt) pentru parole și generarea de token-uri criptate la login.
- **Interacțiuni și Constrângeri:** Colaborează cu toate celelalte servicii prin interceptarea cererilor HTTP. O constrângere majoră este durata de viață a token-ului (60 minute), după care utilizatorul trebuie să se reautentifice.
- **Interfețe:** Exportă metodele verifyToken, generateHash și checkRole. Orice eroare de validare declanșează excepția 401 Unauthorized.

2. Motorul de Matchmaking AI (AIService)

Clasificat ca Serviciu de Logică Business, acesta reprezintă nucleul inteligent al platformei. Definiște procesul prin care preferințele umane sunt comparate cu trăsăturile animalelor.

- **Procesare și Compoziție:** Include subserviciul de Prompt Engineering, care transformă datele brute din MongoDB în text pentru API-ul OpenAI. Algoritmul analizează similaritatea semantică între nevoile adoptatorului și profilul animalului. Procesarea este asincronă, având o complexitate de timp dependentă de latența API-ului extern.
- **Structuri Interne:** Folosește Context Objects (liste de mesaje care păstrează istoricul conversației) pentru a oferi răspunsuri coerente.
- **Interfețe și Volume:** Exportă endpoint-ul /match/analyze. Raportează un volum de date de aproximativ 2-5 KB per interacțiune, cu o limită de 1000 de caractere per mesaj pentru a respecta constrângerile de cost și performanță ale API-ului.

3. Managerul de Resurse și Persistență (DataManager)

Este un Serviciu de Date care gestionează comunicarea cu MongoDB și stocarea imaginilor. Scopul său este să asigure consistența informațiilor și accesul rapid la fișierele media.

- **Procesare:** Utilizează driverul Mongoose pentru a mapa documentele BSON în obiecte JavaScript. Gestionează operațiunile CRUD și curățarea datelor (sanitization) înainte de stocare.
- **Interacțiuni:** Este utilizat intens de AIService pentru a extrage liste de animale și de AuthGateway pentru verificarea credențialelor. Interacționează extern cu AWS S3 pentru a genera URL-uri temporare pentru poze.

- **Constrângeri:** Datele de intrare sunt limitate prin scheme stricte (ex. imagini sub 5MB). În caz de indisponibilitate a bazei de date, serviciul aruncă excepția `DatabaseConnectionError` și activează mecanismul de retry.
- **Raportare:** Gestionează volume mari de date (imagini HD), optimizându-le prin tehnici de compresie la nivel de serviciu înainte de exportul către Frontend.

Proiectare detaliată de securitate

Securitatea sistemului este construită pe principiul „apărării în profunzime”, utilizând următoarele mecanisme:

Autentificare și Autorizare

Accesul este securizat prin JWT (JSON Web Tokens), iar parolele sunt protejate prin hashing bcrypt. Utilizăm un model RBAC (Role-Based Access Control), astfel încât adoptatorii și personalul ONG să aibă acces strict la datele permise rolului lor.

Criptarea Datelor

Informațiile sunt protejate în ambele stări critice:

- În tranzit: Criptare TLS 1.3 (HTTPS) pentru toate comunicațiile externe și interne.
- În repaus: Stocare criptată pe disc folosind standardul AES-256 pentru baza de date și fișierele media.

Rețea și Porturi

Suprafața de atac este minimizată prin deschiderea exclusivă a portului 443 (HTTPS) către internet. Comunicarea cu baza de date (port 27017) se realizează doar în interiorul unei rețele private (VPC), fiind invizibilă din exterior.

Audit și Protecție

Sistemul monitorizează activitatea prin jurnale de audit în AWS CloudWatch, detectând orice acces neautorizat. Protecția activă este asigurată de un WAF (Web Application Firewall) și AWS GuardDuty, care blochează automat atacurile de tip SQL Injection, XSS sau tentativele de forță brută.

Proiectare de detaliu pentru performanța sistemului

Performanța și fiabilitatea sistemului sunt asigurate printr-o arhitectură elastică, menită să gestioneze vârfurile de trafic fără degradarea experienței utilizatorului.

1. Capacitate și Performanță

- **Volum estimat:** Sistemul este dimensionat pentru 1.000+ utilizatori, cu un vârf de 100 de sesiuni simultane. Stocarea este prevăzută pentru ~50 GB de date media (imagini) și ~1 GB pentru date structurate (BSON).
- **Așteptări de performanță:** Timpul de încărcare a paginilor trebuie să fie sub 2 secunde, iar latența răspunsurilor asistentului AI (excluzând procesarea LLM externă) sub 500ms.

- Optimizare: Utilizăm un Content Delivery Network (CDN) pentru livrarea imaginilor, reducând sarcina asupra serverului principal și latența pentru utilizatorii aflați la distanță de centrul de date.

2. Disponibilitate și Fiabilitate (High Availability)

Pentru a atinge o disponibilitate de 99,9%, sistemul elimină punctele unice de eșec (SPOF) prin redundanță:

- Clustering: Serverul de aplicație rulează în mai multe instanțe în spatele unui Load Balancer. Dacă o instanță eșuează, traficul este preluat instantaneu de celelalte.
- Multi-AZ Deployment: Resursele sunt distribuite în cel puțin două zone de disponibilitate (data centers) diferite. Chiar și un incident major la un centru de date nu va opri funcționarea platformei.
- Bază de date redundantă: MongoDB Atlas rulează într-un Replica Set cu 3 noduri; dacă nodul principal cade, un nod secundar este ales automat ca lider în câteva secunde.

3. Backup, Recuperare și Arhivare

- Backup: Se realizează snapshot-uri automate zilnice ale bazei de date și ale stocării de fișiere, cu o perioadă de retenție de 30 de zile.
- Recuperare (Disaster Recovery): În cazul unei coruperi de date, sistemul permite „Point-in-Time Recovery”, restabilind baza de date la starea exactă dintr-un anumit moment al zilei.
- Arhivare: Cererile de adopție finalizate de peste 2 ani sunt mutate automat într-o stocare de tip „Cold Storage” (AWS Glacier) pentru a reduce costurile, rămânând disponibile doar pentru audit.

4. Puncte Critice și Monitorizare

Singurul punct teoretic de eșec este conexiunea către API-ul OpenAI. Pentru acest scenariu, am proiectat un mod de degradare grațioasă: dacă AI-ul este indisponibil, utilizatorul primește un mesaj de tip „mentenanță asistent”, dar poate folosi în continuare filtrele manuale de căutare și formularele standard de adopție. Monitorizarea constantă prin CloudWatch ne alertează în timp real despre orice creștere a ratei de eroare sau a consumului de resurse.

1.1 Proiectare detaliată a comunicațiilor interne (între componente)

Sistemul SmartAdopt funcționează într-o arhitectură virtualizată de tip VPC (Virtual Private Cloud), unde comunicarea între componente este optimizată pentru latență minimă și securitate maximă.

Topologie și Noduri

Utilizăm o topologie de tip Star (Stea) la nivel logic. Punctul central de intrare este un Load Balancer, care distribuie traficul către minimum două Servere de Aplicație (instanțe EC2). Acestea, la rândul lor, comunică cu un cluster de bază de date format din 3 noduri (Replica Set). Deși sunt componente virtuale, ele sunt plasate în zone de disponibilitate diferite pentru a elimina riscurile hardware locale.

Formate de Date și Protocoale

Toate schimburile de informații între componentele interne (Backend -> Bază de date) și externe (Client -> Backend) folosesc formatul JSON. Pentru stocarea efectivă în baza de date, driverul transformă automat datele în BSON (format binar JSON). Comunicarea se realizează prin protocolul HTTPS (TLS 1.3) pentru securitate, utilizând apeluri de tip REST API.

Sincronizare și Control

Nu există un „bus” fizic, fiind un mediu cloud. Sincronizarea este în principal asincronă pentru operațiunile care implică inteligența artificială (pentru a nu bloca interfața utilizatorului) și sincronă pentru tranzacțiile critice în baza de date (cum ar fi finalizarea unei adopții), asigurând integritatea datelor prin mecanisme de „locking” la nivel de rând.

Conectivitate și Distanțe

Componentele sunt localizate în aceeași regiune geografică (de exemplu, Frankfurt), ceea ce reduce distanța virtuală la o latență de rețea de sub 1 milisecundă. Fluxul de date este bidirecțional: cererile pleacă de la utilizator către serverul de aplicație, care interoghează baza de date sau API-ul AI și returnează rezultatul procesat înapoi către client. Această proximitate virtuală elimină nevoia de hardware de rețea complex între componente, totul fiind gestionat de infrastructura de tip Software Defined Networking (SDN) a furnizorului cloud.

Controale pentru verificarea integrității sistemului

Pentru a asigura integritatea totală a datelor, sistemul implementează următoarele mecanisme de control simplificate:

1. Controlul Accesului (RBAC)

Accesul este strict ierarhizat: utilizatorii văd doar ce au nevoie. Adoptatorii își gestionează propriul profil, în timp ce ONG-urile au drept de scriere doar pentru animalele din propria bază. Accesul direct la baza de date este blocat pentru orice entitate din afara backend-ului.

2. Auditare și Jurnalizare Dinamică

Orice interacțiune cu datele critice este înregistrată automat într-o pistă de audit. Fiecare intrare în jurnal conține obligatoriu:

- Identitatea: Cine (ID Utilizator).
- Locația: De unde (IP și Terminal).
- Momentul: Când (Data/Ora ISO).
- Acțiunea: Ce s-a modificat sau accesat. Jurnalurile sunt păstrate 12 luni pentru operațiuni curente și 5 ani pentru audit de management.

3. Validare prin Tabele Standard

Pentru a elimina erorile umane (typos), câmpurile cheie (ex: Specie, Status Adopție, Talie) sunt validate prin tabele de referință. Utilizatorul alege din opțiuni predefinite, nu introduce text liber, garantând astfel consistența raportărilor.

4. Siguranța Operațiunilor (Soft Delete & Verificare)

- Fără ștergeri iremediabile: Se folosește „Soft Delete” (datele sunt ascunse, nu eliminate fizic timp de 30 de zile), permițând recuperarea rapidă în caz de eroare.
- Validare Multi-strat: Orice adăugare sau modificare trece printr-o verificare automată de format, lungime și tip înainte de a fi salvată, prevenind coruperea bazei de

